



Framework Guide

RestAPI Automation Framework — Layer by Layer

PART 1 + OVERVIEW

C# (.NET 8) · RestSharp · Reqnroll (SpecFlow) · FluentAssertions · ExtentReports



Overview — 7 Parts

Yeh guide poore framework ko **7 parts** mein todti hai. Har part ek layer hai, aur har layer agle ko feed karti hai. Neeche se upar padho — sab jud jayega.

Part	Layer	Kya seekhoge
1	Configuration	JSON read/write, <code>ConfigReaderNew</code> , <code>AppConfiguration</code>
2	Hosts & Clients	<code>ApiHost</code> , <code>RestClientFactory</code>
3	Authentication	<code>AuthService</code> , <code>TokenManager</code> , token lifecycle
4	Endpoint Resolution	<code>EndpointHelper</code> , <code>{placeholder}</code> replace
5	Request & Send	<code>RequestBuilder</code> , <code>ApiClient</code> , flexible requests
6	Driver Layer	<code>UserDriver</code> — test actions facade
7	Feature/Steps/Hooks/Validator	Gherkin → C# binding, assertions



End-to-end flow (ek line mein)

```
Feature step → ApiHostStepHelper (host set) → UserDriver (action)
  → ApiClient (verb) → RequestBuilder (request + token) → EndpointHelper (placeholder)
  → RestClientFactory (client) → RestSharp (HTTP) → response
  → TokenContext (save) → ResponseValidator (assert)
```



PART 1 — Configuration Layer

Goal: Samajhna ki framework apni saari values (URLs, credentials, endpoints, tokens, test data) kahan se padhta aur kahan likhta hai.

Yeh layer sabse pehle aati hai kyunki **baaki saari layers isi se data maangti hain** — login credentials, base URLs, endpoint paths, cached IDs, token — sab kuch JSON files mein hai, aur unhe padhne ka kaam yeh layer karti hai.

1.1 — JSON files ka role (data kahan rehta hai)

File	Kya rakhti hai	Example key
appsettings.json	Base URLs, environment, auth mode, file paths, cached IDs, token	ApiUrls , access_token , CacheId
appsettings.QA.json / .Staging.json	Environment-specific overrides	per-env URLs
TestData/Login/LoginRequest.json	Login credentials, role-wise	SuperAdmin , HospitalRole
TestData/Request Endpoint/RequestEndPoint.json	API endpoint paths (placeholders ke saath)	getExistingUser
TestData/Request Endpoint/RequestBody.json	POST/PUT request bodies	productCreateBody

appsettings.json ka anatomy (zaroori keys)

```
{
  "Environment": "QA",
  "ApiUrls": {
    "AuthBaseUrl": "https://testRovicare.b2clogin.com/", // login server
    "ApiBaseUrl": "https://as-rc-test-api.azurewebsites.net/" // app server
  },
  "Authentication": {
    "Mode": "Shared", // Shared ya PerScenario
    "PersistAccessToken": true, // token wapas file mein likhna hai?
    "UseCachedAccessToken": true
  },
  "EndpointJson": "TestData/Request Endpoint/RequestEndPoint.json", // endpoints file ka path
  "LoginJson": "TestData/Login/LoginRequest.json", // creds file ka path
  "LoginRoleKey": "HospitalRole", // default role
  "access_token": "eyJhbGc ... ", // last login ka token (framework khud likhta hai)
  "CacheId": "2c528cd8- ... ", // last response se nikla cached id
}
```

```
"EmailId": "hospital@rovicare.com"
}
```

💡 **Insight:** `appsettings.json` ke andar **paths bhi keys hain** — `EndpointJson` , `LoginJson` . Matlab framework pehle `appsettings.json` padhta hai, usme se doosri file ka path nikaalta hai, fir woh file padhta hai. (Indirection — yeh aage Part 4/5 mein dikhega.)

1.2 — ConfigReaderNew (main hero of this layer)

📄 `Core/Configurations/ConfigReaderNew.cs`

Yeh ek **static, thread-safe, file-based** config loader hai. Iska design rule:

Ek time pe ek file "active" rehti hai. Jab tum nayi file `LoadConfig` karte ho, purani memory se dispose ho jaati hai.

(a) `LoadConfig(fileName)` — file ko active banao

```
ConfigReaderNew.LoadConfig("appsettings.json");
```

- File ko memory mein load karta hai aur "default" active config bana deta hai.
- Purani saari entries dispose kar deta hai (clean slate).

(b) `GetValue(key)` — active file se value

```
string email = ConfigReaderNew.GetValue("EmailId"); // "hospital@rovicare.com"
string url = ConfigReaderNew.GetValue("ApiUrls:ApiBaseUrl"); // nested → ':' use karo
```

- Nested key ke liye colon (`:`) — jaise `"Authentication:Mode"` .
- Agar key nahi mili to **empty string** deta hai (null nahi).

(c) `GetJsonBody(file, key)` — poora nested block string mein

```
// LoginRequest.json mein "SuperAdmin" ek poora object hai
string creds = ConfigReaderNew.GetJsonBody("TestData/Login/LoginRequest.json",
"SuperAdmin");
// creds =
{"grant_type":"password","client_id":" ... ","username":"admin@ ... ","password":" ... "}
```

💡 Yeh `GetValue` se alag hai — `GetValue` flat string deta hai, `GetJsonBody` **poora JSON sub-tree** deta hai. Login body banane ke liye yahi use hota hai.

(d) `UpdateJsonValue(file, key, value)` — file mein WAPAS likhna

```
ConfigReaderNew.UpdateJsonValue("appsettings.json", "access_token", newToken);
```

- Login ke baad token isi se `appsettings.json` mein persist hota hai.

(e) `GetSection<T>(name)` — strongly-typed binding

```
var urls = ConfigReaderNew.GetSection<ApiEnvironmentSettings>("ApiUrls");  
// urls.AuthBaseUrl, urls.ApiBaseUrl
```

Aur bhi (reference)

Method	Kaam
<code>ReadJson<T>(file)</code>	File ko <code>T</code> object mein deserialize (disk se, cache nahi)
<code>ReadJson(file)</code>	Mutable <code>JsonNode</code> tree
<code>WriteJson<T>(file, data)</code>	Object ko JSON file mein likho
<code>UpdateJsonSection(file, section, dict)</code>	Nested section update/create (e.g. <code>SessionInfo</code>)
<code>GetJsonValue(file, key)</code>	File se directly top-level value (cache bypass)
<code>GetJsonSectionValue(file, section, prop)</code>	Nested section se ek value

1.3 — Path resolution ka jaadu (kahan se file dhoondta hai)

Jab tum `"appsettings.json"` doge (relative path), `ConfigReaderNew` is order mein dhoondta hai:

- `.csproj` ke paas (source tree) — `FindFileNearProjectRoot`
- `bin/Debug/ ...` (output folder) — `AppContext.BaseDirectory`
- **Current working directory**

🗨 **Kyun?** Taaki jab framework token wapas `appsettings.json` mein likhe, woh **source tree** wali file update ho (jo tum IDE mein dekhte ho) — sirf `bin` wali copy nahi.

1.4 — AppConfigurration (layered, read-only config)

📄 `Core/Configurations/AppConfiguration.cs`

`ConfigReaderNew` "ek file at a time" model hai. `AppConfiguration` iske ulat — **layered + cached** model deta hai, jo startup pe ek baar build hota hai:

```
appsettings.json
+ appsettings.{Environment}.json  (optional override)
+ Environment variables (prefix "ROVI_")
```

Environment ka naam yahan se aata hai (priority order):

- `TEST_ENVIRONMENT` environment variable
- `appsettings.json` ka `"Environment"` key
- default `"QA"`

Key members

```
AppConfiguration.ApiUrls;           // → ApiEnvironmentSettings (AuthBaseUrl, ApiBaseUrl,
Timeout)
AppConfiguration.EnvironmentName;    // "QA" / "Staging"
AppConfiguration.Authentication;    // → AuthenticationSettings (Mode,
PersistAccessToken ... )
AppConfiguration.FeatureBaseUrlMap; // feature name → host map
AppConfiguration.GetValue("BaseUrl");
AppConfiguration.GetBool("UseCachedAccessToken", true);
AppConfiguration.Reload();           // cache clear karke dobara build
```

ApiEnvironmentSettings (bound object)

📄 `Core/Configurations/ApiEnvironmentSettings.cs`

```
public sealed class ApiEnvironmentSettings
{
```

```

public string AuthBaseUrl { get; set; } = "";
public string ApiBaseUrl { get; set; } = "";
public int TimeoutMilliseconds { get; set; } = 30_000;
}

```

Yeh `appsettings.json` ke `"ApiUrls"` + `"Timeout"` se bind hota hai. Part 2 mein `RestClientFactory` isi ko use karke clients banata hai.

1.5 — Do config classes kyun? (ConfigReaderNew vs AppConfigurration)

	ConfigReaderNew	AppConfiguration
Model	Ek file at a time (swap-able)	Layered (merged), cached
Read	✓	✓
Write	✓ (UpdateJsonValue, WriteJson)	✗ (read-only)
Env overrides	✗	✓ (appsettings.{env}.json, env vars)
Use case	Test data / creds / endpoints / token persist	Base URLs, auth mode, framework settings

 **Rule of thumb:** Dynamic test data ya likhna hai → `ConfigReaderNew`. Framework-level settings (URLs, mode) chahiye → `AppConfiguration`.

1.6 — Mini walkthrough: "SuperAdmin ke credentials kaise milte hain?"

```

// 1. appsettings.json active karo
ConfigReaderNew.LoadConfig("appsettings.json");

// 2. usme se creds-file ka PATH nikaalo
string loginFile = ConfigReaderNew.GetValue("LoginJson");
// → "TestData/Login/LoginRequest.json"

// 3. ab us file se "SuperAdmin" ka poora block lo
string credsJson = ConfigReaderNew.GetJsonBody(loginFile, "SuperAdmin");
// →
{"grant_type":"password","client_id":"...","username":"admin@rovicare.com","password":"..."}

// 4. ab isse LoginRequest object banega (yeh Part 3 mein)

```

Bas yahi indirection pattern poore framework mein baar-baar dikhega: **appsettings** → **path key** → **doosri file** → **actual value**.

✓ Part 1 Summary

- **JSON files** = saara data (URLs, creds, endpoints, bodies, token, cached IDs).
- **ConfigReaderNew** = ek-file-at-a-time read/write engine. `LoadConfig` → `GetValue` / `GetJsonBody` / `UpdateJsonValue` core methods.
- **AppConfiguration** = layered + cached + env-aware, read-only framework settings.
- **ApiEnvironmentSettings** = base URLs ka typed model.
- **Indirection pattern**: `appsettings.json` mein doosri files ke paths hote hain.

➡ **Next:** Part 2 — ApiHost & RestClientFactory ("kaunsa base URL, kaunsa client").